Lab 7 - APIfying AI

Part 1

When should the service return 400 vs 404?
It should return 400 when you entered Invalid input (bad JSON or out-of-bounds/wall start/end) and 404 when no path was found like „http://127.0.0.1:5000".

Compare the specification to the models within models.py. What similarities can you identify?
Both describe the shape of the data (the fields, types, wether something is required/optional for our services)

What might be the reason behind using both the OpenAPI specification and the Pydantic models compared to just using one of them?

The openAPI specification does not enforce something but makes the API clear to the outside world by giving an overview on how our requests/responses should look like.

The pydantic models make the api clear to the inside of our code. Running the api without it is possible but could, for example, lead to a crash of the program when a client sends wrong data etc

Part 2

if start = end, what should /solve return?
Return [1,1] as it is still a valid path.

How could this service support future algorithms without breaking clients?
Add additional query parameter. If client is not providing anything, we can implement a default search algorithm being used to not brake the program.

What real-world examples use microservices like this?
Navigation applications. Image generation —> Find best path from one pixel to another one.

If we swap BFS for A* internally, what changes should clients see? How do we avoid breaking them?

A client shouldn't see any changes. The API contract stays the same. If changes are needed, use versioning to avoid breaking clients.

How would this service scale to many concurrent agents? What changes (if any) would you make?

Put it behind a production server. Limit payload sizes (e.g. when sending custom mazes). Add observability (logs, metrics). Use caching for repeated solves.

Why include ?explain=true if other agents (including LLM agents) could "just read the JSON" anyhow?

Acts as a low friction bridge (adds a pedagogical explanation of what just happened). Makes an LLM prefer our explanation from the micro service instead of creating its own explanations.